

Credit Name: CSE 3010 Computer Science 3

Assignment Name: BreakAPlate

The following questions can help you in thinking critically about your problem-solving processes:

Understanding the Problem

How did you approach understanding the challenge? Prior to beginning this assignment, I had gone through and completed all the skill builders in this chapter to relearn my earlier concepts of Java and build an understanding of the new GUI aspect of coding. I made sure that I thoroughly comprehended each major part, such as creating GUI functions and integrating them into the project in a seamless manner. Before this chapter, I was unfamiliar with creating components that operated outside of the console, thus posing an unfamiliar challenge to me.

Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion? At the time of this chapter, I was absent for a majority of the lecture and in-class demonstrations. This was due to my major health issues that arose. When I began working on the Mastery projects, the GUI took much more time than I had anticipated, greatly due to the lack of knowledge that was taught during class time. As a result, I was unaware of the use of the "Design" tab in Eclipse that had to be imported into the software, and would make a whole lot of difference when constructing my GUI's. My classmate helped me resolve this confusion and personally taught me how to import and install this feature.

Planning the Solution

Did you create a plan or break the problem into smaller steps before coding? I approached this project with simply a plan constructed in my head. I had decided to begin with the exteriors of the program, or in other words, the GUI. Before beginning this step, I had looked over the class example in the D2L shelf, and there, I determined the necessary amount of buttons and elements I would need to include in my GUI. After creating the GUI, I created the script in the main method and wrote the code for how the game would progress. For this, I began by scripting the start and end buttons, then moving on to the breaking plate portion, and finally creating the part of the code which determines the winner and resets the game. I thought that creating the project in chronological order made the most sense and would allow me to identify and revise any errors as they arose throughout the process.

How did you decide on the tools, data structures, or I decided that I wanted to take on more than the assignment called for, and adjusted my program to allow the user to break each individual plate instead of simply throwing all balls at once. With this in mind, I had to adjust my program and include methods that would do certain things when pressed. For example, for each plate, I made a method that

algorithms to use?	determines and displays the result of each ball thrown, and a method to calculate the winning status of the game. I also implemented a reset button and some visual cues when hitting each plate to further allow the user to see the game progression and enhance visuals. Similar to past projects, I implemented a string and a boolean to work as signs if the game was ongoing, stopped, or completed. When a certain event was completed, each variable would be adjusted according to the action.
Implementation	
Did you write the code in small pieces or attempt the entire solution at once?	As stated in an earlier segment, I wrote the code in small pieces, according to each major step of the program. I meticulously went chronologically and wrote the code top to bottom in accordance with the progression of the program. In the end, this method worked great for me, and even allowed me to handle errors that wouldn't make or break my whole program when adjusted.
How did you test your solution along the way to make sure it was working?	Throughout the project, I tested my code every couple of lines to make sure my program was still functioning correctly. If something went wrong or there was a small bug, I would either block out some of the written code with “//” s to separate and identify the issue at hand, or use print statements in the console to find the problem. After completing the project, I made sure to go in and test every possible combination of buttons and possible human actions and accounted for things that broke the program. I then went in and adjusted my code until those issues were resolved.
Overcoming Challenges	
What part of the problem was the most difficult for you?	The part of the project that I found most difficult was where I had to create extra motion cues and text that would flash during each throw and attempt to break a plate. I ran into numerous issues with this extra part. The largest issue I had was creating a part in the code that would only display the text for a set amount of time before disappearing. When I first began to create this part, I attempted to use a try and catch loop with the sleep function. This did not work because it would halt the entire program, not just stop the text from appearing. Instead, I researched numerous other methods on scripting forums and determined the best way to approach this in a GUI was to use a built-in timer in Java. I had to first create a timer task object, and then run it as a timer. This would allow my text to disappear after a certain amount of time without stopping or freezing the program at all.
How did you handle moments when you felt stuck or unsure of what to do next?	During moments where I felt stuck or unsure of what to do next, I opted to look online on scripting forums, such as Stack Overflow, to find other individuals who had similar issues to mine and adjust my code according to their fixes. If I had any problems with how the code was supposed to be structured, or how GUI components were supposed to be scripted, I heavily relied on the textbook and looked through the examples listed.

Learning	
Was there anything you learned that you think will help you with future challenges?	I believe that the concept of GUI will be very important throughout my journey of learning Java, as most programs rely on the external GUI to process data back to the user and function as a standalone program. The contents of this chapter will definitely aid me in the future chapters and further build the foundation I have for GUI in my future studies. For this project specifically, I believe that my discovery and learning of how to tackle my timer issue will be needed in my future projects if I wish to do something similar. I believe that important things like that allow us to integrate more ideas in future work and expand on our knowledge and understanding outside of what is written and taught in the textbook.

SCRIPT OVERVIEW:

(Please note: the following code is not in order of how it is presented in the Java file, but rather in the order of how I scripted it. There may also be certain areas of the code that are not present here as they do not contribute to the overall main algorithms, but rather are limited to method headers, error methods [automatically generated when making the file] or brackets.)

```
// Initialize variables and GUI components
private ImageIcon plate, broken1, broken2, broken3, tiger, sticker ;
private JButton plate1, plate2, plate3, button;
private JLabel status, prize, p1Stat, p2Stat, p3Stat;
private JFrame frmBreakaplate;
private String gameStat = "off";
private Boolean win;
private int shoot, shape, counter;
List<ImageIcon> plates = new ArrayList<>();
Timer timer = new Timer();
```

When starting this project, I first began by initializing the variables that I needed and imagined relevant in my code. For some variables like JButtons or JLabels, Eclipse automatically imported these variables as I used the “Design” feature to establish my GUI.

```
private void initialize()
{
    // Initialize image icons
    plate = new ImageIcon("../Chapter10/src/Mastery/plate.jpg");
    broken1 = new ImageIcon("../Chapter10/src/Mastery/broken1.jpg");
    broken2 = new ImageIcon("../Chapter10/src/Mastery/broken2.jpg");
    broken3 = new ImageIcon("../Chapter10/src/Mastery/broken3.jpg");
    tiger = new ImageIcon("../Chapter10/src/Mastery/tiger.gif");
    sticker = new ImageIcon("../Chapter10/src/Mastery/sticker.gif");

    //Main JFrame
    frmBreakaplate = new JFrame();
    frmBreakaplate.setTitle("BreakAPlate");
    frmBreakaplate.setBounds(100, 100, 400, 379);
    frmBreakaplate.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    //JPanels
    JPanel panel = new JPanel();
    panel.setBackground(new Color(232, 247, 255));
    frmBreakaplate.getContentPane().add(panel, BorderLayout.CENTER);
    panel.setLayout(null);
}
```

Please note that not all initialized GUI components are shown because of the large number of them. This is just a snippet.

Next, I created my GUI on the design page and initialized the components. I made sure to name each element in a clear manner to avoid mixups. I also decided to initialize the images as image icons so that later on I could quickly swap between images on my buttons and headers. I made sure to include an active listener on each button.

```
//When a button is pressed
public void actionPerformed(ActionEvent e)
{
    //Set button's action command to variable "eventName"
    String eventName = e.getActionCommand();

    // If button's event name is equal to "play"
    if(eventName.equals("play"))
    {
        // Set all plates an action commands to unbroken plates
        plate1.setIcon(plate);
        plate1.setActionCommand("shoot1");

        plate2.setIcon(plate);
        plate2.setActionCommand("shoot2");

        plate3.setIcon(plate);
        plate3.setActionCommand("shoot3");

        // Change button text + action command to stop
        button.setText("Stop");
        button.setForeground(Color.red);

        button.setActionCommand("stop");

        // Turn game on
        gameStat = "on";

        // Set counter to 0
        counter = 0;

        // Set winner to true
        win = true;

        // Change status text
        status.setText(" Click the plates to shoot!");
    }
}
```

```
// If button's event name is equal to "stop"
if(eventName.equals("stop"))
{
    // Reset button visuals
    button.setForeground(new Color(44, 150, 104));
    button.setActionCommand("play");
    button.setText("Play");

    // Reset plate icons
    plate1.setIcon(plate);
    plate2.setIcon(plate);
    plate3.setIcon(plate);

    prize.setIcon(null);

    // Reset game counters / win stat
    counter = 0;
    win = true;
    status.setText(" Press: [Play] to start!");
}

// If first plate is clicked
if(eventName.equals("shoot1"))
{
    // Perform method "hit" with the corresponding plate
    hit(plate1, p1Stat);
}

else if (eventName.equals("shoot2"))
{
    hit(plate2, p2Stat);
}

else if (eventName.equals("shoot3"))
{
    hit(plate3, p3Stat);
}
}
```

I then created a method to determine actions after a button is pressed and an action command is given. Certain action commands can only be registered and completed if other conditions are met. At the beginning of the game, if the start button is pressed, the program adjusts all plate images to unbroken ones, changes the “play” button to a different “stop” button if the user wants to end the game mid session, resets the counter to 0, officially starts the game by setting the variable “gameStat” to “on”, and change “win” variable to true. The “win” variable changes during the gameplay to determine an outcome according to the number of plates hit. If a ball is thrown, but the plate is unbroken, then the variable will be adjusted to false, outputting a losing prize when the game is finished. After the game is started, the plate buttons can now be clicked. Each plate will have its action command reset.

Once the game is activated, the other buttons are able to be pressed. If the user presses the “stop” button (originally the play button), the program will revert that button back into a play button and reset all original conditions of the game before it started. During gameplay, each plate can be clicked once, and an outcome about the throw is randomly calculated by a method called “hit”, which changes text stats, plate images, and the “win” variable according to the outcome. Each plate can only be clicked once because once it is clicked on one time, the action command is changed, only to be reverted when the whole game is reset.

```

//Method to determine hit outcome & output winner if found
public void hit(JButton a, JLabel b)
{
    //If the game status is equal to on
    if (gameStat.equals("on"))
    {
        //Add 1 throw to the counter
        counter++;
        //Generate a random number from 1-2 to determine hit outcome
        shoot = (int)(2*Math.random() + 1);
        // If random number is equal to 1
        if (shoot == 1)
        {
            //Generate a random number 1-3 to select broken plate icon
            shape = (int)(3*Math.random());
            //Set plate's icon to chosen icon
            a.setIcon(plates.get(shape));
            //Change hit status color to green
            b.setForeground(new Color(44, 150, 104));
            //Set text to reflect outcome
            b.setText(" HIT!");
            //After 0.5 seconds, clear status label
            timer.schedule(new TimerTask()
            {
                public void run()
                {
                    b.setText(" ");
                }
            }, 500);
        }
        // If random number is equal to 2
        else
        {
            // Set hit status color to red
            b.setForeground(Color.red);
            // Set text to reflect outcome
            b.setText("MISS!");
            // After 0.5 seconds, clear status label
            timer.schedule(new TimerTask()
            {
                public void run()
                {
                    b.setText(" ");
                }
            }, 500);
        }
        // If plate is not successfully hit, change win stat to false
        win = false;
    }
}

```

This method is used to calculate the outcome when a ball is thrown to break a plate. A number is generated from 1-2. If the number is 1, then the plate will be set to its broken state, and the corresponding status text will be changed to show the result.

If the random number is 2, then the plate will remain unbroken, the status text will be changed, and the win variable will be set to false. This means that when the game is over, the losing outcome will be displayed.

```

// If all 3 plates are attempted
if(counter == 3)
{
    // If all plates are successfully hit
    if(win == true)
    {
        // Change button text color + change text to reflect win
        button.setForeground(new Color(44, 150, 104));
        button.setText("YOU WIN!");
        // Change prize text
        status.setText("prize: tiger");
        // After 1.2 seconds, change button text and color
        timer.schedule(new TimerTask()
        {
            public void run()
            {
                button.setForeground(Color.black);
                button.setText("Reset");
            }
        }, 1200);
        // Change prize icon to reflect outcome
        prize.setIcon(tiger);
    }
    // If one or more plates are not successfully hit
    else
    {
        // Change button text color + change text to reflect loss
        button.setText("YOU LOOSE!");
        status.setText("prize: sticker");
        // After 1.2 seconds, change button text and color
        timer.schedule(new TimerTask()
        {
            public void run()
            {
                button.setForeground(Color.black);
                button.setText("Reset");
            }
        }, 1200);
        // Change prize icon to reflect outcome
        prize.setIcon(sticker);
    }
}
}

```

After all plates are attempted to be broken, the prize image will be changed to the result of the “win” boolean.